DATA 202 — Quiz 4

ANSWER KEY — Version B

Question 1 (SLO 06A)

Distances from the new loan: 1 (late), 1.41, 2, 2.24 (three on time), 2.83 (late), 3 (on time), 4.24 (late).

- **k = 1: late** (the single nearest loan was late).
- **k = 5: on time** (the 5 nearest: 3 on time, 2 late).
- **k = 1 is more likely to overfit**: it follows single training points (noise), so it is perfect on the data it learned from but less reliable on new loans.

Question 2 (SLO 06B)

Metric	Fraction	Value
Accuracy	(134 + 20) / 200	0.770
Precision (late)	20 / (20 + 13) = 20/33	0.606
Recall (late)	20 / (20 + 33) = 20/53	0.377
Lazy model	147 / 200	0.735

Accuracy 77.0% beats the lazy model's 73.5% only a little, and recall shows the model catches fewer than half of the late loans.

Question 3 (SLO 06C)

1. **learn:** `model.fit(X_train, y_train)`
2. **predict:** `model.predict(X_test)`
3. **score:** compare the predictions with `y_test` → accuracy

Test set: **160** loans (800 × 0.2).

The model learns from the training rows only. The test rows play the part of new loans: the model sees X_test only to predict, and y_test only to check its answers.

Question 4 (SLO 07A)

prior_late	pages	class_year	prediction
1	300	3	**on time**
4	320	2	**late**
2	250	1	**on time**

`class_year` is never asked by this tree, so it doesn't matter. Each loan follows one path from the top question to a leaf.

Question 5 (SLO 07B)

max_depth = 2 (best CV score, 0.770). The training score keeps rising to 0.935 with no limit because a deep tree memorizes the folds it was fit on; on held-out folds it falls to 0.710 (overfitting). **Underfits:** depth 1 (poor on both rows).

Question 6 (SLO 07C)

```
grid = {"max_depth": [2, 3, 5],
 "min_samples_leaf": [5, 20, 40]}
search = GridSearchCV(DecisionTreeClassifier(), grid, cv=5)
search.fit(X_train, y_train)
search.best_params_   # {'max_depth': 5, 'min_samples_leaf': 20}
search.best_score_    # 0.783
search.score(X_test, y_test)   # 0.76
```

9 combinations × 5 folds = **45 trees** (plus one final refit of the winner on all the training loans).

Why 0.76 < 0.783: `best_score_` is the **best of 9** cross-validation scores, so the winner was partly lucky on these folds (the best of many is optimistic). The test set is new data that played no part in choosing, so its score is the honest one (and it is one split).

Question 7 (SLO 04A) — Retake from Quiz 3

Table	Column	Answer
`loans`	`book_id`	Foreign key -> books
`books`	`book_id`	Primary key
`students`	`prior_late`	Neither (a measurement, not an identifier)
`loans`	`student_id`	Foreign key -> students (it repeats: a student can borrow many books)
`loans`	`loan_id`	Primary key

Question 8 (SLO 04B) — Retake from Quiz 3

Result	how=	rows
1	**left**	4
2	**outer**	5
3	**inner**	3
4	**right**	4

S023 and S024 appear in both tables (3 loans): that's **inner**. **Left** keeps every loan, so L201 (student S021, not in `students`) appears with class_year NaN. **Right** keeps every student, so S022 (no loans) appears with loan_id NaN. **Outer** keeps both, so it has both NaN rows.

Left and right both give 4 rows here, so the row count alone can't tell them apart: look at **which side has the NaN**.

Question 9 (SLO 04C) — Retake from Quiz 3

```
loans_long = pd.melt(
    monthly_loans,
    id_vars=["student_id"],
    value_vars=["sep", "oct", "nov"],
    var_name="month",
    value_name="n_loans",
)
```

student_id	month	n_loans
S002	sep	0
S002	oct	1
S002	nov	1
S005	sep	1
S005	oct	0
S005	nov	1

6 rows: 2 wide rows × 3 month columns. Row order may differ (pd.melt lists all sep rows first, then oct, then nov); any order is fine.